

Relatório Técnico: Análise Empírica de Desempenho de Algoritmos de Ordenação

1. Introdução ao Problema

A ordenação de dados constitui uma das operações fundamentais na Ciência da Computação, sendo um componente crítico em sistemas que variam desde bancos de dados de larga escala até mecanismos de busca e processamento de sinais. A eficiência de um sistema de software frequentemente depende da escolha apropriada de um algoritmo de ordenação, a qual deve ser fundamentada tanto na análise assintótica teórica quanto em evidências experimentais.

Enquanto a análise assintótica permite prever o comportamento de um algoritmo à medida que o tamanho da entrada (n) tende ao infinito, a análise empírica revela o impacto de constantes ocultas, o comportamento da hierarquia de memória e a sensibilidade do algoritmo à distribuição dos dados. O presente relatório documenta a implementação e a avaliação de três algoritmos clássicos (Quick Sort, Heap Sort e Radix Sort) com o objetivo de confrontar suas complexidades teóricas com o desempenho observado em um ambiente computacional real.

2. Descrição dos Algoritmos Implementados

2.1. Quick Sort

O Quick Sort é um algoritmo baseado no paradigma de "dividir para conquistar". A implementação realizada utiliza o **esquema de partição de Lomuto**, no qual o último elemento do vetor é selecionado como pivô para organizar os demais elementos em duas partições: uma com valores menores ou iguais ao pivô e outra com valores maiores.

Em termos de complexidade, o algoritmo apresenta desempenho médio e de melhor caso de $O(n \log n)$. Entretanto, a estratégia de Lomuto é particularmente sensível à organização prévia dos dados; em vetores já ordenados ou inversamente ordenados, o particionamento torna-se altamente desequilibrado, induzindo uma degradação do desempenho para $O(n^2)$ e aumentando significativamente o consumo de memória na pilha de execução devido à profundidade da recursão.

2.2. Heap Sort

O Heap Sort fundamenta-se na utilização de uma estrutura de dados de árvore binária completa denominada heap. Para esta implementação, foi adotado o **Max-Heap**, onde a raiz sempre contém o maior elemento da estrutura. O processo consiste em duas etapas: a construção inicial do heap (fase de *build-max-heap*) e a extração sucessiva do elemento máximo, seguida pela reorganização da estrutura (*heapify*).

A principal vantagem técnica do Heap Sort é a sua consistência. Ele garante uma complexidade de tempo de $O(n \log n)$ em todos os cenários (melhor caso, médio e pior caso). Além disso, é um algoritmo *in-place*, exigindo apenas O de memória auxiliar, o que o torna

extremamente robusto para sistemas com restrições de recursos, apesar de possuir constantes de tempo ligeiramente superiores às do Quick Sort em distribuições aleatórias.

2.3. Radix Sort

Diferente dos anteriores, o Radix Sort é um algoritmo de ordenação não comparativo. Ele processa os números dígito a dígito, da posição menos significativa (LSD) para a mais significativa. Para que o algoritmo funcione corretamente, é imperativo que a ordenação de cada dígito seja realizada por uma **sub-rotina estável**; conforme os requisitos deste projeto, o **Counting Sort** foi a técnica empregada para essa finalidade.

A complexidade teórica do Radix Sort é de $O(d(n + k))$, onde d é o número de dígitos e k é a base do sistema numérico (neste caso, base 10). Em termos práticos, para chaves inteiras em um intervalo limitado, o algoritmo tende a apresentar comportamento linear $O(n)$, superando algoritmos de comparação em grandes volumes de dados, embora sua performance seja influenciada pela necessidade de memória adicional para o vetor auxiliar da sub-rotina Counting Sort.

3. Metodologia Experimental

A avaliação empírica foi estruturada para garantir a reprodutibilidade e a precisão dos dados coletados. O experimento seguiu as seguintes diretrizes:

- **Ambiente de Execução:** O código foi compilado em padrão C11 utilizando otimizações de nível 3 (-O3) para refletir o desempenho em cenários de produção.
- **Amostragem de Dados:** Foram utilizados 21 tamanhos de entrada, variando de $n=10$ até $n=1.000.000$ elementos.
- **Cenários de Distribuição:** Para cada tamanho, foram gerados três tipos de vetores:
 - **Aleatório:** Inteiros não negativos gerados aleatoriamente.
 - **Crescente:** Vetor já ordenado de 1 a n .
 - **Decrescente:** Vetor ordenado de forma inversa, de n a 1.
- **Instrumentação de Tempo:** Utilizou-se a função `clock_gettime()` com o parâmetro `CLOCK_MONOTONIC`, garantindo precisão de nanossegundos e isolando o tempo de processamento das tarefas de I/O ou alocação de memória. Cada valor registrado representa a média de 5 execuções consecutivas para mitigar ruídos do sistema operacional.
- **Segurança de Pilha:** Devido à natureza recursiva do Quick Sort com partição de Lomuto, implementou-se uma trava de segurança para interromper a execução em cenários de pior caso ($O(n^2)$) acima de 25.000 elementos, evitando o erro de *stack overflow*. Tais casos são marcados nos resultados com o valor -1.0.

4. Apresentação dos Resultados

Os tempos de execução (em segundos) para os principais marcos do experimento estão sintetizados na tabela abaixo:

Tabela 1: Desempenho em Cenário Aleatório (Caso Médio)

Tamanho (n)	Quick Sort (s)	Heap Sort (s)	Radix Sort (s)
10.000	0,001021	0,001452	0,000920
100.000	0,010895	0,018109	0,009383
500.000	0,059357	0,113234	0,056413
1.000.000	0,128332	0,276911	0,114935

Tabela 2: Desempenho em Cenário Crescente (Pior Caso para Quick Sort)

Tamanho (n)	Quick Sort (s)	Heap Sort (s)	Radix Sort (s)
5.000	0,015836	0,000491	0,000351
10.000	0,061335	0,001004	0,000917
25.000	-1,000000	0,002725	0,002305

